DESENVOLVIMENTO MOBILE · MÓDULO 07 · TRILHA FLUTTER · NÍVEL INICIANTE

Estado, Navegação e Animações

setState, ciclo de vida, elevação de estado, ChangeNotifier, ciclo de vida do app, Navigator, go_router, diálogos e animações

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Diferenciar estado efêmero de estado do app e usar `setState` corretamente.
- Explicar o ciclo de vida de um `State` e do próprio app.
- Elevar o estado e compartilhá-lo com `ValueNotifier` e `ChangeNotifier`.
- Navegar entre telas com `Navigator` e com **go_router** (rotas, parâmetros, abas e proteção por login).
- Mostrar diálogos, *snackbars* e *bottom sheets*.
- Criar animações implícitas, explícitas e transições com `Hero`.

Roteiro

Neste módulo

- Fundamentos: estado, ciclo de vida e navegação
- Estado
- Navegação
- Animações
- Fechamento

Fundamentos: estado, ciclo de vida e navegação

Estado: o que a tela precisa lembrar

Estado

Todo valor que pode mudar com o tempo e influencia o que a tela mostra. A interface é uma função dele: **UI = f(estado)**.

Tipo	Exemplo no LabMonitor	Onde mora
Efêmero	painel expandido, rolagem, aba ativa	no componente
Da tela	bancadas carregadas, filtro, erro	no dono da tela
Do app	usuário logado, tema escolhido	em um serviço compartilhado
Persistente	reservas, preferências	banco ou disco (Módulo 10)

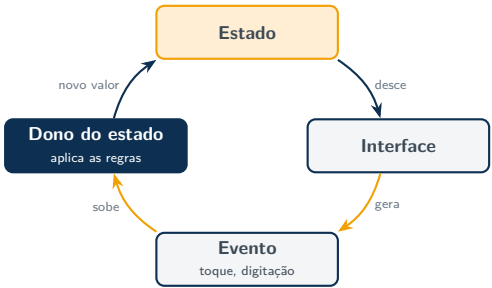
Estado derivado

Valor calculado a partir de outros, como o total de bancadas livres. Não se guarda: calcula-se, para nunca ficar desatualizado.

Fonte única da verdade

Cada dado tem um único dono que pode alterá-lo; os demais apenas leem. Duas cópias editáveis do mesmo dado acabam divergindo.

Fluxo unidirecional de dados



Fluxo unidirecional (UDF)

O estado desce do dono até a interface; os eventos sobem da interface até o dono. **Só o dono altera o estado**.

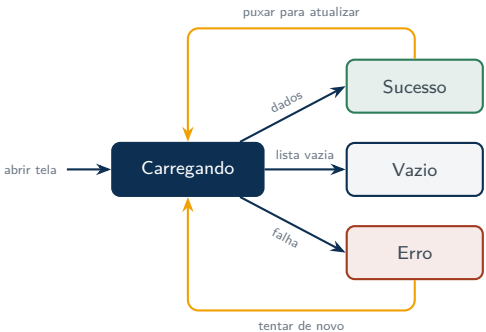
Vantagens

Previsível: cada mudança tem um único caminho. Fácil de depurar: registre eventos e estados. Fácil de testar: um evento entra, um estado sai.

Um pouco de história

A Arquitetura Elm, o Flux do Facebook (2014) e o Redux (2015) popularizaram o modelo, hoje padrão no Compose, no Flutter e no SwiftUI.

A tela como máquina de estados



Máquina de estados finitos

Conjunto fechado de estados e de transições permitidas, disparadas por eventos. A tela está sempre em **exatamente um** estado.

Estados impossíveis

Três booleanos (`carregando`, `erro`, `vazio`) geram 8 combinações, a maioria sem sentido. Uma classe selada com quatro casos admite só os quatro válidos.

Dica

Kotlin e Dart 3 têm classes seladas: o compilador obriga o `when` ou o `switch` a tratar todos os casos.

O que sobrevive a quê

Onde o valor está	nova recomposição	rotação da tela	morte do processo	app fechado pelo usuário
variável local dentro da função de interface	não	não	não	não
`remember` (Compose)	**sim**	não	não	não
`rememberSaveable` (Compose)	**sim**	**sim**	**sim**	não
`ViewModel` (Android)	**sim**	**sim**	não[1]	não
`State` de um `StatefulWidget` (Flutter)	**sim**	**sim**[2]	não	não
`RestorationMixin` (Flutter)	**sim**	**sim**	**sim**	não
banco de dados, arquivo, preferências (Módulo 10)	**sim**	**sim**	**sim**	**sim**

[1] sim, com `SavedStateHandle`. [2] o Flutter não recria a tela ao girar.

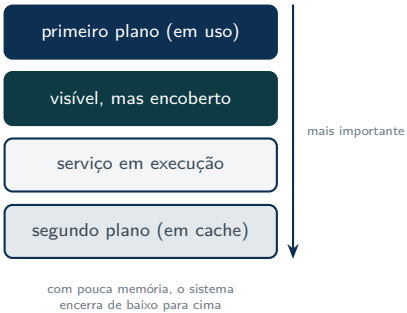
Morte do processo

Em segundo plano, o sistema pode encerrar o app para liberar memória. Ao voltar, o usuário espera a tela como a deixou: o sistema a recria e devolve o pequeno estado salvo.

Como testar

Ative "Não manter atividades" nas opções do desenvolvedor do Android, ou rode `adb shell am kill <pacote>` com o app em segundo plano.

Ciclo de vida e prioridade dos processos



Ciclo de vida

Sequência de estados por que passam o app e cada tela, controlada pelo sistema: criado, visível, em primeiro plano, em segundo plano, destruído. Cada transição é avisada por *callbacks*.

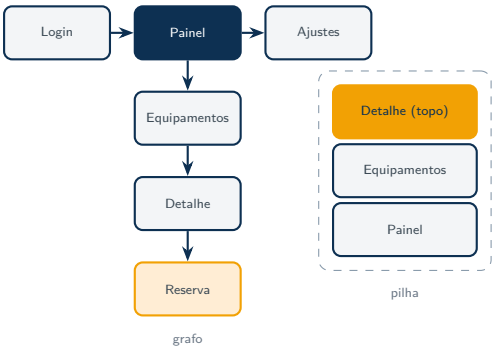
O que fazer nas transições

Ao sair do primeiro plano: pare câmera, sensores e animações e salve rascunhos. Ao voltar: retome e atualize os dados.

Por que o sistema encerra apps

A memória do celular é limitada, e o sistema prefere encerrar apps em segundo plano a deixar o aparelho lento: no Android, o *low memory killer*; no iOS, o *jetsam*.

Navegação: grafo de destinos e pilha de retorno



Grafo e pilha

Destinos (telas) são **nós**; as ligações permitidas, **arestas**. A pilha de retorno é LIFO: navegar empilha, voltar desempilha.

Princípios

Destino inicial fixo; o voltar nunca surpreende; cada aba da barra inferior tem a própria pilha.

Voltar preditivo

Com `targetSdk` 36 no Android 16, o gesto mostra a tela anterior antes de o dedo soltar e `onBackPressed` não é mais chamado: use `BackHandler` ou `PopScope`.

Navegação declarativa e links profundos

Imperativa *versus* declarativa

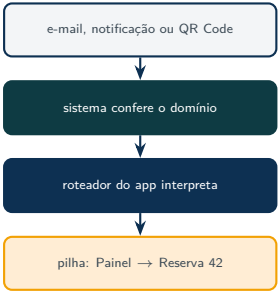
Imperativa: comandos `push` e `pop`. Declarativa: a pilha é um **estado** (uma lista de destinos) e o que aparece é função dela, como na interface (Navigation 3, go_router).

Link profundo (*deep link*)

Endereço que abre o app direto em um destino, como `https://labmonitor.app/reserva/42`. O app monta uma pilha sintética (Painel, Reserva) para o voltar fazer sentido.

Atenção

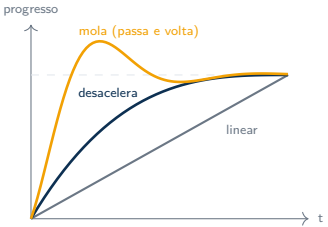
Parâmetros de um link vêm de fora: valide-os (a reserva existe? este usuário pode vê-la?).



Links verificados

App Links (Android, arquivo `assetlinks.json`) e *Universal Links* (iOS, `apple-app-site-association`) provam que o site e o app têm o mesmo dono.

Movimento: interpolação, curvas e molas



Interpolação

A cada quadro: `valor = início + (fim − início) × curva(t)`, com `t` indo de 0 a 1 ao longo da animação.

Curvas e molas

Curvas de aceleração (*easing*) imitam objetos reais: desaceleram ao chegar. **Molas** são definidas por rigidez e amortecimento, não por duração, e podem ser interrompidas sem saltos. O Material 3 Expressive (2025) adotou molas como base.

Movimento com propósito

Anime para orientar: de onde veio a tela, o que mudou. Transições de 150 a 400 ms mantêm o app ágil.

Acessibilidade

Respeite a opção do sistema de remover ou reduzir animações: movimento excessivo causa desconforto em algumas pessoas.

Estado

Estado efêmero e estado do app

Estado efêmero (local)

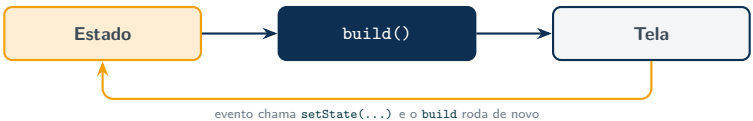
Pertence a um único widget e pode ser perdido sem problema: aba selecionada, campo expandido, progresso de uma animação.

Ferramenta: `StatefulWidget` + `setState`.

Estado do app (compartilhado)

Usado por várias telas ou que precisa sobreviver: usuário logado, lista de bancadas, reservas, preferências.

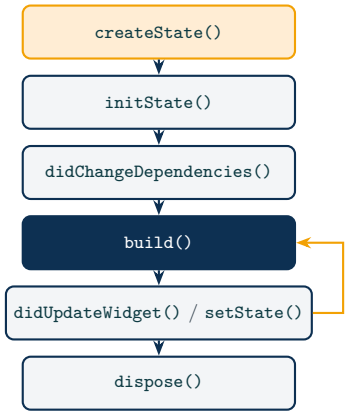
Ferramenta: `ChangeNotifier`, *view models* e gerência de estado (Módulo 08).



Dica

Comece com estado local e eleve-o só quando outra parte do app precisar dele.

Ciclo de vida de um `State`



Método	Use para
`initState`	criar controladores, iniciar inscrições (uma vez)
`didChangeDependencies`	reagir a `Theme`, `MediaQuery`, `Provider`
`build`	descrever a tela (rápido, sem efeitos)
`didUpdateWidget`	o pai mudou os parâmetros
`dispose`	cancelar e liberar tudo

Atenção

`build` pode rodar a cada quadro. Nada de requisições ou cálculos pesados dentro dele.

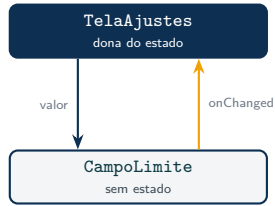
Elevando o estado

Componente sem estado

```dart
class CampoLimite extends StatelessWidget {
  const CampoLimite({super.key,
      required this.valor, required this.onChanged});
  final double valor;                    // estado desce
  final ValueChanged<double> onChanged; // evento sobe

  @override
  Widget build(BuildContext context) => Slider(
    value: valor, min: 20, max: 60, divisions: 40,
    label: '${valor.round()} °C',
    onChanged: onChanged,
  );
}

// o pai é o dono do estado
CampoLimite(
  valor: _limite,
  onChanged: (v) => setState(() => _limite = v),
);
```



Fluxo unidirecional

O estado desce como parâmetro e os eventos sobem como funções. Há uma única fonte da verdade.

ValueNotifier e ChangeNotifier

Estado observável

```
class CarrinhoReservas extends ChangeNotifier {
  final _itens = <Reserva>[];
  List<Reserva> get itens => List.unmodifiable(_itens);

  void adicionar(Reserva r) {
    _itens.add(r);
    notifyListeners();          // avisa quem ouve
  }
}

final carrinho = CarrinhoReservas();

ListenableBuilder(
  listenable: carrinho,
  builder: (context, _) =>
      Badge(label: Text('${carrinho.itens.length}'),
            child: const Icon(Icons.event)),
);
```

Um único valor

```
final tema = ValueNotifier(ThemeMode.system);

ValueListenableBuilder(
  valueListenable: tema,
  builder: (context, modo, _) =>
      MaterialApp(themeMode: modo, ...),
);
tema.value = ThemeMode.dark;   // reconstrói
```

Conceito

Só o `builder` é reconstruído quando o objeto notifica. É a base dos *view models* recomendados pela documentação do Flutter.

Ciclo de vida do app

Pausando o sensor em segundo plano

```dart
class _MonitorState extends State<Monitor> {
  late final AppLifecycleListener _ouvinte;

  @override
  void initState() {
    super.initState();
    _ouvinte = AppLifecycleListener(
      onResume: () => sensor.retomar(),
      onPause: () => sensor.pausar(),
    );
  }

  @override
  void dispose() {
    _ouvinte.dispose();
    super.dispose();
  }
}
```

Estado	Significado
`resumed`	visível e recebendo toques
`inactive`	visível, sem foco (chamada, multitarefa)
`hidden`	não visível
`paused`	em segundo plano
`detached`	motor ativo sem tela

Morte do processo

Em segundo plano o sistema pode encerrar o app. Salve no banco ou no disco o que o usuário não pode perder (Módulo 10).

Navegação

Navigator: a pilha de telas

Navegação imperativa

```dart
// abrir uma tela
Navigator.of(context).push(MaterialPageRoute(
  builder: (_) => TelaDetalhe(id: eq.patrimonio),
));

// voltar devolvendo um resultado
Navigator.of(context).pop(true);

// esperar o resultado
final salvou = await Navigator.of(context).push<bool>(
    MaterialPageRoute(builder: (_) => const TelaNova()));
if (salvou == true) recarregar();
```

Dica

Ótimo para fluxos simples. Para apps com URLs, *deep links*, abas e login, use o **go_router**.



Conceito

O botão ou gesto voltar do sistema faz `pop`. `PopScope` permite interceptar (ex.: "descartar alterações?").

go_router: rotas declarativas

lib/routing/router.dart

```dart
final router = GoRouter(
  initialLocation: '/painel',
  refreshListenable: auth,                  // reavalia o login
  redirect: (context, state) =>
      auth.logado ? null : '/login',        // proteção
  routes: [
    GoRoute(path: '/login',
        builder: (_, __) => const TelaLogin()),
    GoRoute(path: '/painel',
        builder: (_, __) => const TelaPainel()),
    GoRoute(path: '/equipamentos',
      builder: (_, __) => const TelaEquipamentos(),
      routes: [
        GoRoute(path: ':id', builder: (_, state) =>
            TelaDetalhe(id: state.pathParameters['id']!)),
      ]),
  ],
);

MaterialApp.router(routerConfig: router);
```

Chamada	Efeito
`context.go('/painel')`	troca a pilha pela rota
`context.push('/x')`	empilha por cima
`context.pop()`	volta
`context.go('/equipamentos/42')`	caminho com parâmetro

Por que go_router

Pacote mantido pela equipe do Flutter: URLs na web, *deep links* (abrir o app num link), proteção de rotas e navegação por abas.

Abas com estado preservado

StatefulShellRoute

```
StatefulShellRoute.indexedStack(
  builder: (context, state, shell) => Scaffold(
    body: shell,
    bottomNavigationBar: NavigationBar(
      selectedIndex: shell.currentIndex,
      onDestinationSelected: shell.goBranch,
      destinations: const [
        NavigationDestination(
            icon: Icon(Icons.dashboard), label: 'Painel'),
        NavigationDestination(
            icon: Icon(Icons.memory), label: 'Equipamentos'),
        NavigationDestination(
            icon: Icon(Icons.settings), label: 'Ajustes'),
      ]),
  ),
  branches: [
    StatefulShellBranch(routes: [rotaPainel]),
    StatefulShellBranch(routes: [rotaEquipamentos]),
    StatefulShellBranch(routes: [rotaAjustes]),
  ],
);
```

Uma pilha por aba

Cada `StatefulShellBranch` guarda sua própria navegação: ao voltar para a aba Equipamentos, o usuário reencontra o detalhe que estava aberto.

Diálogos e avisos

`showDialog`, `showModalBottomSheet` e `showSnackBar` (via `ScaffoldMessenger`) também são camadas sobre a tela atual.

Animações

Animações implícitas

Basta mudar o valor

```
AnimatedContainer(
  duration: const Duration(milliseconds: 300),
  curve: Curves.easeOut,
  height: expandido ? 160 : 72,
  color: emAlerta ? Colors.orange : Colors.white,
  child: conteudo,
);

AnimatedSwitcher(                    // troca com fade
  duration: const Duration(milliseconds: 250),
  child: Text('$temperatura °C',
      key: ValueKey(temperatura)),
);

TweenAnimationBuilder<double>(
  tween: Tween(begin: 0, end: nivel),
  duration: const Duration(seconds: 1),
  builder: (_, v, __) => LinearProgressIndicator(value: v),
);
```

Implícita

Você muda uma propriedade e o widget anima sozinho do valor antigo ao novo. Cobre a maioria dos casos.

Widget	Anima
`AnimatedContainer`	tamanho, cor, borda
`AnimatedOpacity`	transparência
`AnimatedAlign`	posição
`AnimatedSwitcher`	troca de filho
`TweenAnimationBuilder`	qualquer valor

Animações explícitas e Hero

Indicador pulsando

```
class _AlertaState extends State<Alerta>
    with SingleTickerProviderStateMixin {
  late final _ctrl = AnimationController(vsync: this,
      duration: const Duration(seconds: 1))..repeat(reverse:
      true);
  late final _escala = Tween(begin: 1.0, end: 1.3)
      .animate(CurvedAnimation(parent: _ctrl,
          curve: Curves.easeInOut));

  @override
  void dispose() { _ctrl.dispose(); super.dispose(); }

  @override
  Widget build(BuildContext context) => ScaleTransition(
      scale: _escala,
      child: const Icon(Icons.warning, color: Colors.orange));
}
```

Explícita

Um `AnimationController` controla o tempo (iniciar, repetir, inverter). Use quando precisar de laços, sequências ou controle fino.

Hero

Envolva a foto do equipamento em `Hero(tag: eq.patrimonio, ...)` na lista e no detalhe: ela "voa" entre as telas.

Pacotes

`animations`, **Rive** e **Lottie**.

Fechamento

Resumo

Resumo

- Estado efêmero com `setState`; estado compartilhado com `ChangeNotifier` e *view models*.
- `initState` cria, `dispose` libera; `build` só descreve a tela.
- Eleve o estado: valor desce, evento sobe, uma fonte da verdade.
- `AppLifecycleListener` pausa recursos em segundo plano.
- go_router: rotas declarativas, parâmetros, `redirect` para login e abas com `StatefulShellRoute`.
- Animações implícitas para o dia a dia; `AnimationController` para controle fino; `Hero` entre telas.

Glossário (1/2)

Termo	Significado
Estado	valor que muda com o tempo e afeta o que a tela mostra
Estado efêmero	pertence a um widget e pode se perder sem prejuízo
Estado derivado	valor calculado a partir de outros; não se guarda
Fonte única da verdade	cada dado tem um único dono que pode alterá-lo
Fluxo unidirecional	o estado desce até a interface; os eventos sobem até o dono
`setState`	avisa que o estado mudou e agenda um novo `build`
`initState/dispose`	início e fim do ciclo de vida de um `State`
`ChangeNotifier`	objeto que avisa seus ouvintes quando muda
`AppLifecycleState`	estado do app: `resumed`, `inactive`, `hidden`, `paused`, `detached`

Glossário (2/2)

Termo	Significado
Morte do processo	encerramento do app em segundo plano para liberar memória
`RestorationMixin`	salva e devolve o estado depois da morte do processo
Pilha de retorno	histórico LIFO dos destinos visitados
go_router	roteador declarativo baseado em caminhos de URL
Link profundo	endereço que abre o app direto em um destino
`PopScope`	controla o voltar, inclusive o gesto preditivo do Android
`Curve`	função de aceleração de uma animação
`AnimationController`	controla tempo e direção de uma animação explícita

Questões de revisão

1. Por que um contador guardado em uma variável dentro de `build` não funciona?
2. Explique o fluxo unidirecional com um campo de busca e um `ChangeNotifier`.
3. Modele a tela de equipamentos como máquina de estados. Por que uma `sealed class` e não três booleanos?
4. O que acontece com o `State` de um widget quando o Android encerra o processo em segundo plano? Como restaurar?
5. Que recursos liberar quando o app passa para `paused`?
6. Desenhe a pilha depois de `push` de Equipamentos e de Detalhe a partir do Painel, e de um `pop`.
7. Como o go_router trata o link `/reserva/42`? Que validações fazer?
8. Por que preferir uma mola a uma curva de duração fixa quando o usuário pode interromper o gesto?

Exercícios

Exercício 1 — Ciclo de vida

Registre um `debugPrint` em cada método do ciclo de vida de um `State`. Navegue, gire a tela e volte; descreva a sequência.

Exercício 2 — Navegação

Configure o go_router com login, três abas e a rota `/equipamentos/:id`. Abra o app pelo navegador já na URL de um equipamento.

Exercício 3 — Animação

Faça o cartão da bancada mudar de cor com `AnimatedContainer` quando a temperatura passar do limite e pulsar um ícone de alerta.

LabMonitor — Etapa 7

Implemente a navegação completa: login, Painel, Equipamentos (lista → detalhe → reserva) e Ajustes, com abas preservadas e animação de alerta.

Para aprofundar

- Estado efêmero e de app:
  `https://docs.flutter.dev/data-and-backend/state-mgmt/ephemeral-vs-app`
- Navegação: `https://docs.flutter.dev/ui/navigation`
- go_router: `https://pub.dev/packages/go_router`
- Animações: `https://docs.flutter.dev/ui/animations`
- Ciclo de vida do app:
  `https://api.flutter.dev/flutter/widgets/AppLifecycleListener-class.html`

Respostas das questões de revisão

1. `build` roda de novo a cada reconstrução e a variável volta ao valor inicial. O estado deve ficar em um campo do `State`, alterado com `setState`, ou em um `ChangeNotifier`.
2. O campo mostra o texto vindo do *notifier* (estado que desce) e chama `buscar(texto)` ao digitar (evento que sobe); o *notifier* filtra a lista, chama `notifyListeners` e a tela se reconstrói.
3. Estados `Carregando`, `Sucesso(itens)`, `Vazio` e `Erro(msg)`. A classe selada só admite esses quatro e o `switch` obriga a tratar todos; booleanos permitem combinações impossíveis.
4. Ele se perde: o processo recomeça do zero. Com `RestorationMixin` e `restorationScopeId`, os valores registrados são salvos e devolvidos; dados importantes também podem ir para o disco.
5. Câmera, sensores, fluxos de localização e animações contínuas; também é hora de salvar rascunhos. Tudo é retomado em `resumed`.
6. Depois dos `push`: Painel, Equipamentos, Detalhe (topo). Depois do `pop`: Painel, Equipamentos.
7. A rota `/reserva/:id` recebe `42` em `state.pathParameters`, e rotas aninhadas deixam o Painel abaixo na pilha. Validar se a reserva existe e se o usuário pode vê-la, com `redirect` para o login se preciso.
8. A mola parte da posição e da velocidade atuais e segue a física até o novo alvo; a curva recomeça com duração fixa e muda a velocidade de forma brusca.

Fim do Módulo 07

Próximo: Módulo 08 — Arquitetura e gerência de estado: MVVM, repositórios, Provider, Riverpod e Bloc.